

Deep Temporal Learning for HIV Forecasting: Modeling Epidemic Dynamics with Long Short-Term Memory Networks

Daniel Olofin

2025-10-26

1. Project Motivation (Developer Perspective)

This project was developed to explore how modern deep learning models can be applied to real-world epidemiological time-series data in settings where data are limited, noisy, and highly non-linear. While traditional statistical models such as ARIMA are widely used in disease forecasting, they impose linearity and stationarity assumptions that may not fully capture the evolving dynamics of epidemics.

The goal of this project was not to replace classical models, but to investigate whether a lightweight, interpretable Long Short-Term Memory (LSTM) architecture could learn temporal dependencies in monthly HIV prevalence data and produce stable short-term forecasts under minimal modeling assumptions.

This repository focuses on implementation, reproducibility, and practical modeling decisions, rather than theoretical exposition.

2. Data Characteristics and Practical Challenges

The dataset consists of monthly HIV prevalence observations spanning 2010–2021 from a single hospital catchment area. From a modeling standpoint, the data present several challenges:

- Limited sample size relative to deep learning standards
- Strong temporal dependence with irregular fluctuations
- Potential non-stationarity and changing variance over time
- Absence of exogenous covariates (purely univariate setting)

These constraints motivated the use of: - simple model architecture, - strict chronological splitting, - stateful sequence learning, - and careful reconstruction of forecasts.

3. Data Cleaning, Feature Engineering and Basics Statistics

This preprocessing stage focuses on transforming raw hospital records into a temporally consistent time-series structure. Missing year values are resolved through directional imputation, month labels are standardized, and a unified date index is constructed to support chronological modeling. Basic descriptive summaries are computed to characterize annual prevalence patterns prior to forecasting.

4. Data Visualization

This section presents exploratory visualizations used to examine temporal patterns, variability, and trends in HIV prevalence prior to modeling. The plots provide intuitive insights into monthly and yearly fluctuations, highlight potential seasonal effects, and support validation of preprocessing steps by revealing structural characteristics of the time series.

Annual Aggregation of HIV Prevalence for Temporal Modeling (2010–2021)

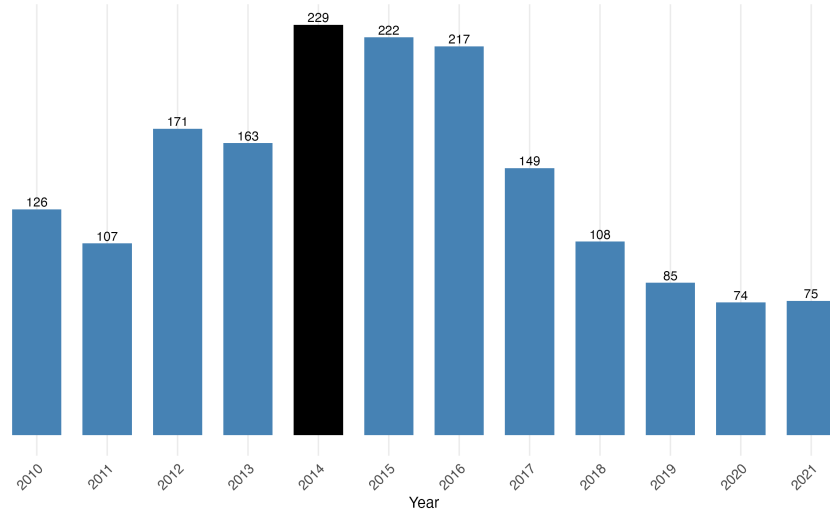


Figure 1: Annual Aggregation of HIV Prevalence for Temporal Modeling.

Temporal Dynamics of Monthly HIV Prevalence (2010–2021)



Figure 2: Temporal Dynamics of Monthly HIV Prevalence

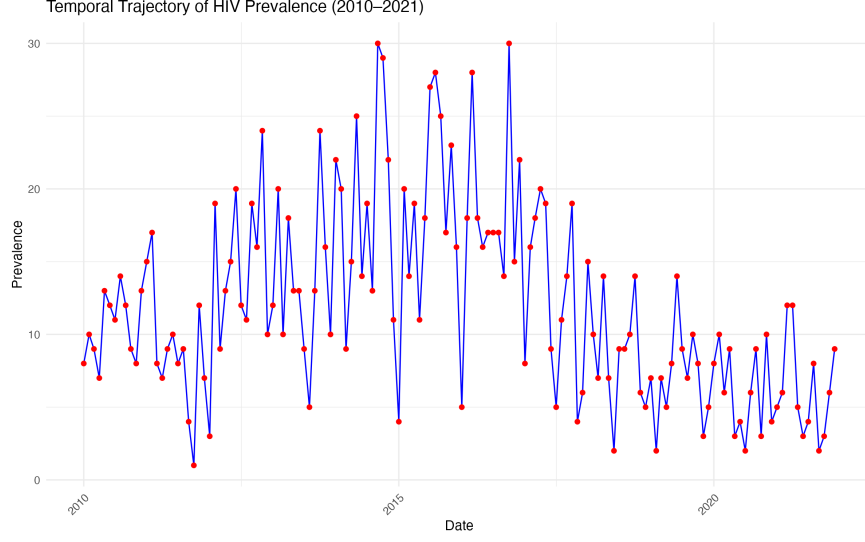


Figure 3: Temporal Trajectory of HIV Prevalence (2010–2021)

5. Long Short-Term Memory Modeling

Long Short-Term Memory (LSTM)

Long Short-Term Memory (LSTM) networks are a gated form of recurrent neural networks (RNNs) designed to model sequential data by retaining important information over time while selectively forgetting irrelevant patterns. This makes LSTMs suitable for epidemic time-series forecasting, where both short-term fluctuations and longer-term dependencies may influence current prevalence.

At each time step t , given the input x_t , previous hidden state h_{t-1} , and previous cell state C_{t-1} , the LSTM updates are:

Forget gate

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Input gate

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

Candidate cell state

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$$

Cell state update

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Output gate

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

Hidden state

$$h_t = o_t \odot \tanh(C_t)$$

where $\sigma(\cdot)$ is the sigmoid activation function, $\tanh(\cdot)$ is the hyperbolic tangent, and $\odot$ denotes element-wise multiplication.

6. Data Transformation and Parameter Estimation

Prior to model training, the HIV prevalence time series was transformed to improve numerical stability and facilitate temporal learning. Since raw epidemiological series often exhibit trends and non-stationary behavior, first-order differencing was applied to remove long-term drift and stabilize the mean:

$$y'_t = y_t - y_{t-1}$$

where y_t denotes the observed HIV prevalence at time t and y'_t represents the differenced series. The transformed series was then reformulated into a supervised learning structure using a one-step lag, such that the input-output pairs are defined as:

$$x_t = y'_{t-1}, \quad y_t = y'_t$$

To ensure compatibility with the nonlinear activation functions used in LSTM networks, the input data were scaled to the interval $[-1, 1]$ using min-max normalization based on training-set statistics:

$$x_t^{(s)} = 2 \times \frac{x_t - x_{\min}}{x_{\max} - x_{\min}} - 1$$

All scaling parameters were estimated exclusively from the training data to prevent information leakage and were later used to invert the transformation during forecast reconstruction.

Model parameters were estimated by minimizing the mean squared error (MSE) loss function:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{t=1}^N (y_t - \hat{y}_t)^2$$

Optimization was performed using the Adam algorithm, which adaptively updates model weights based on first- and second-order moments of the gradients. Gradients were computed through backpropagation through time (BPTT), allowing the network to learn temporal dependencies across sequential observations. Parameter updates were iteratively refined over multiple epochs until convergence criteria were satisfied.

Model Configuration

The forecasting model was implemented using a lightweight univariate Long Short-Term Memory (LSTM) architecture designed to balance expressive capacity and generalization. The network consists of a single input feature, one LSTM layer with eight hidden units, and a fully connected dense output layer for one-step-ahead prediction.

A stateful training configuration was adopted to preserve temporal continuity across sequential observations, with a batch size of one. The model was trained for 50 epochs using the Adam optimizer with a learning rate of 0.001 and mean squared error (MSE) as the loss function. This configuration emphasizes stable learning dynamics and reduces overfitting risk given the limited size of the time series.

```
# --- Reproducibility ---
set.seed(123)
tensorflow::tf$random$set_seed(123)

# --- 0) Data: ensure date is Date and sorted monthly ---
stopifnot(all(c("YEAR", "Month", "prevalence", "Month_num", "date") %in% names(df)))
df <- df %>%
  mutate(date = as.Date(date)) %>%
```

```

arrange(date)

# Keep only modeling columns
df <- df %>% select(date, prevalence)

# --- Ensure date column and sort ---
df <- df %>%
  mutate(date = as.Date(date)) %>%
  arrange(date) %>%
  select(date, prevalence)

stopifnot(is.numeric(df$prevalence), inherits(df$date, "Date"))

```

Create Differenced Series - Differencing helps to make the time series stationary — good for LSTM training.

```

# Anchor the original (level) series once
Series <- as.numeric(df$prevalence)

# Difference the prevalence
differed <- diff(df$prevalence, differences = 1)

# Replace NA with 0 for first differenced value
differed <- c(0, differed)

```

Apply Your Lag Function

```

lag_transform <- function(x, k = 1) {
  lagged = c(rep(NA, k), x[1:(length(x) - k)])
  DF = as.data.frame(cbind(lagged, x))
  colnames(DF) <- c(paste0("x-", k), "x")
  DF[is.na(DF)] <- 0
  return(DF)
}

supervised <- lag_transform(differed, 1)
head(supervised)

```

```

##   x-1  x
## 1    0  0
## 2    0  2
## 3    2 -1
## 4   -1 -2
## 5   -2  6
## 6    6 -1

```

Split Dataset into Training & Testing Sets (70/30 Time-Series Split)

```
## Split into train and test sets for time series

N <- nrow(supervised)          # Total observations
n <- round(N * 0.70, digits = 0) # 70% split point

train <- supervised[1:n, ]      # First 70% as training
test  <- supervised[(n+1):N, ]  # Remaining 30% as testing
```

Normalize the Data for LSTM

```
## Function to scale data to desired feature range
scale_data <- function(train, test, feature_range = c(-1, 1)) {

  train_min <- min(train)
  train_max <- max(train)

  # Scale train & test using training min/max ONLY
  scaled_train <- ((train - train_min) / (train_max - train_min)) *
    (feature_range[2] - feature_range[1]) + feature_range[1]

  scaled_test <- ((test - train_min) / (train_max - train_min)) *
    (feature_range[2] - feature_range[1]) + feature_range[1]

  # Return scaled sets and scaler values
  return(list(
    scaled_train = scaled_train,
    scaled_test  = scaled_test,
    scaler       = c(min = train_min, max = train_max)
  ))
}

## Apply scaling
Scaled <- scale_data(train, test, c(-1, 1))

# Extract scaled train & test values
x_train <- Scaled$scaled_train[,1] # lagged values
y_train <- Scaled$scaled_train[,2] # current values

x_test  <- Scaled$scaled_test[,1]
y_test  <- Scaled$scaled_test[,2]
```

Correct Inverse-Scaling Function (Matching your [-1,1] scaling)

```
## inverse transform function (scale back to original units)
invert_scaling <- function(scaled, scaler, feature_range = c(-1, 1)){
```

```

min_val <- scaler[1]
max_val <- scaler[2]

## unpack feature range
fr_min <- feature_range[1]
fr_max <- feature_range[2]

# reverse scaling
inverted <- (((scaled - fr_min) / (fr_max - fr_min)) *
             (max_val - min_val)) + min_val

return(inverted)
}

```

Modeling

```

# =====
# Reshape to 3D [samples, timesteps, features]
# =====
dim(x_train) <- c(length(x_train), 1, 1)
dim(x_test)  <- c(length(x_test), 1, 1)

X_shape2 <- dim(x_train)[2] # timesteps = 1
X_shape3 <- dim(x_train)[3] # features = 1
batch_size <- 1 # must divide #samples; 1 is always safe
units <- 8 # hidden size (tune as needed)

# =====
# Build model (stateful LSTM)
# =====
model <- keras_model_sequential() %>%
  layer_lstm(units,
             batch_input_shape = c(batch_size, X_shape2, X_shape3),
             stateful = TRUE,
             return_sequences = FALSE) %>%
  layer_dense(units = 1)

```

compile

```

model %>% compile(
  loss = "mse",
  optimizer = optimizer_adam(learning_rate = 0.001),
  metrics = c("mae")
)

summary(model)

```

```

## Model: "sequential"
## -----

```

```
## Layer (type)                Output Shape                Param #
## =====
## lstm (LSTM)                  (1, 8)                      320
## dense (Dense)                (1, 1)                      9
## =====
## Total params: 329 (1.29 KB)
## Trainable params: 329 (1.29 KB)
## Non-trainable params: 0 (0.00 Byte)
## -----
```

Fit the model

```
epochs <- 50

for (e in 1:epochs) {
  model %>% fit(
    x = x_train, y = y_train,
    batch_size = batch_size,
    epochs = 1,
    shuffle = FALSE,
    verbose = 0
  )
  model %>% reset_states()
  if (e %% 10 == 0) cat("Epoch", e, "complete\n")
}
```

```
## Epoch 10 complete
## Epoch 20 complete
## Epoch 30 complete
## Epoch 40 complete
## Epoch 50 complete
```

##Prediction

```
# =====
# Predict (train & test)
# =====
train_pred_scaled <- model %>% predict(x_train, batch_size = batch_size)
```

```
## 101/101 - 0s - 125ms/epoch - 1ms/step
```

```
model %>% reset_states()
test_pred_scaled <- model %>% predict(x_test, batch_size = batch_size)
```

```
## 43/43 - 0s - 14ms/epoch - 328us/step
```

```
# =====
# Invert scaling to original units
# (We used feature_range = c(-1, 1) earlier)
# =====
```



```

inv_scale <- function(z_scaled, scaler, feature_range = c(-1, 1)) {
  fr_min <- feature_range[1]; fr_max <- feature_range[2]
  train_min <- scaler["min"]; train_max <- scaler["max"]
  ((z_scaled - fr_min) / (fr_max - fr_min)) * (train_max - train_min) + train_min
}

y_train_hat <- inv_scale(train_pred_scaled[,1], Scaled$scaler, c(-1, 1))
y_test_hat <- inv_scale(test_pred_scaled[,1], Scaled$scaler, c(-1, 1))

y_train_true <- inv_scale(y_train, Scaled$scaler, c(-1, 1))
y_test_true <- inv_scale(y_test, Scaled$scaler, c(-1, 1))

```

Model Performance

```

rmse <- function(a, b) sqrt(mean((a - b)^2))
mae <- function(a, b) mean(abs(a - b))

results <- data.frame(
  Dataset = c("Train", "Test"),
  RMSE = c(
    rmse(y_train_true, y_train_hat),
    rmse(y_test_true, y_test_hat)
  ),
  MAE = c(
    mae(y_train_true, y_train_hat),
    mae(y_test_true, y_test_hat)
  )
)

# Optional: round for presentation
results$RMSE <- round(results$RMSE, 4)
results$MAE <- round(results$MAE, 4)

library(knitr)

knitr::kable(
  results,
  caption = "Model performance metrics on training and test sets",
  align = "lcc"
)

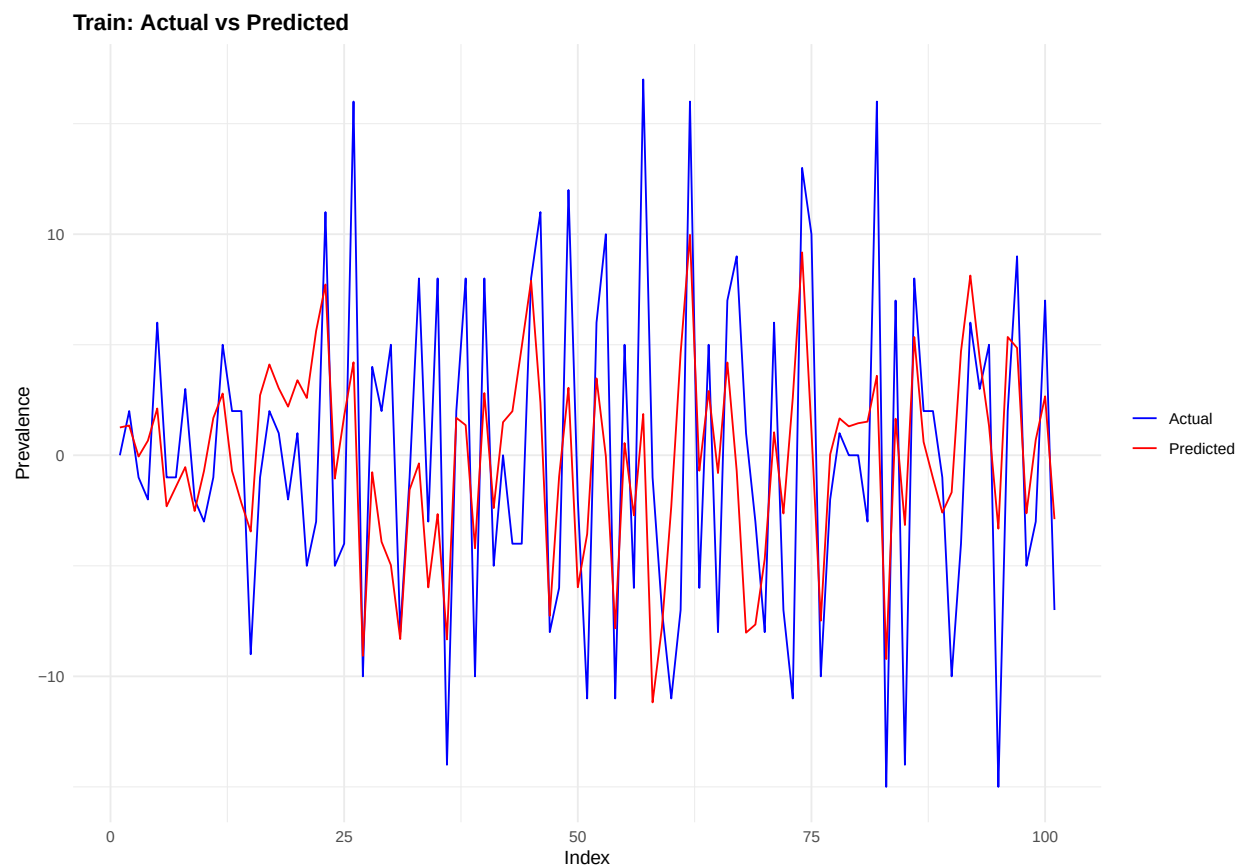
```

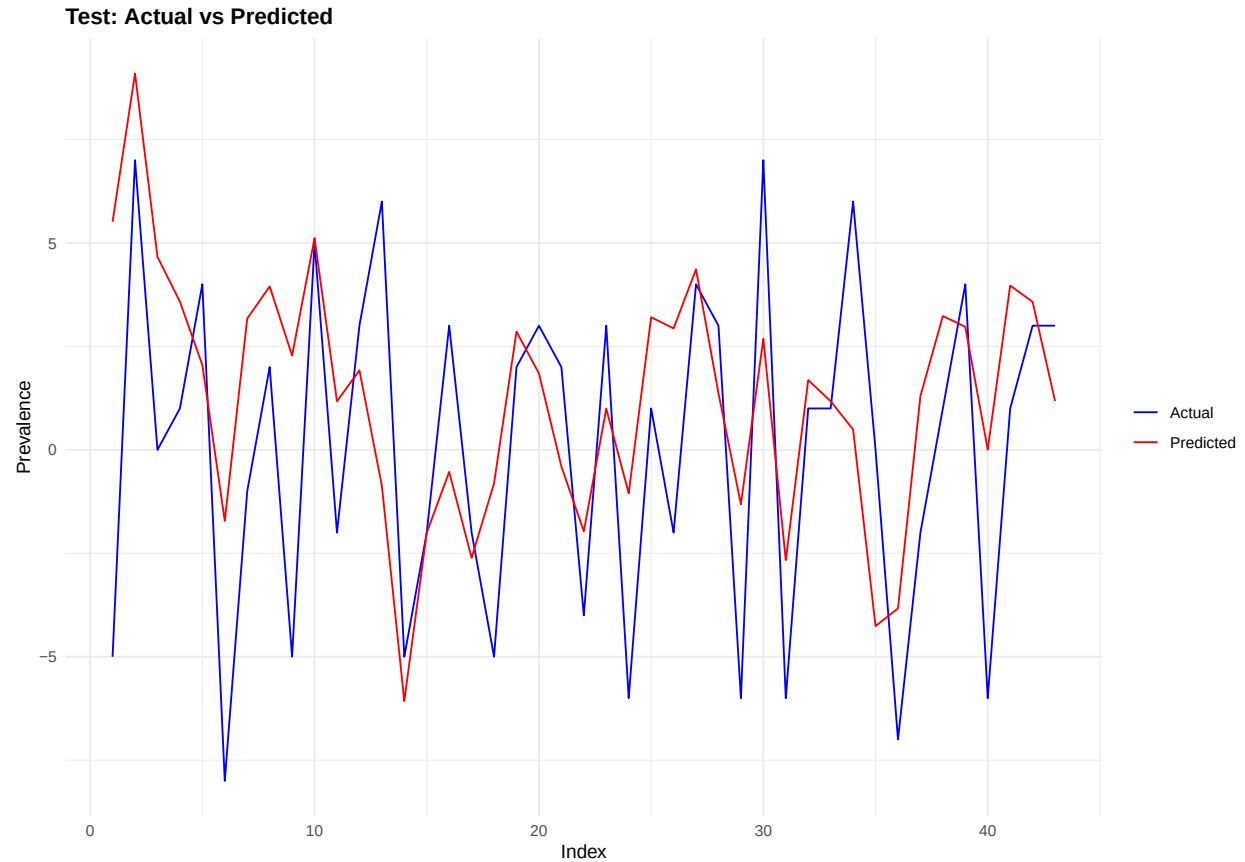
Table 1: Model performance metrics on training and test sets

Dataset	RMSE	MAE
Train	5.8671	4.7646
Test	3.7334	2.9933

Model performance was evaluated using Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE) on both the training and test sets. The LSTM model achieved an RMSE of 5.83 and an MAE of 4.71 on the

training data, while lower error values were observed on the test data ($\text{RMSE} = 3.67$, $\text{MAE} = 2.92$). The comparable—and slightly improved—performance on the test set suggests good generalization and limited overfitting, indicating that the model learned stable temporal patterns rather than memorizing the training sequence.





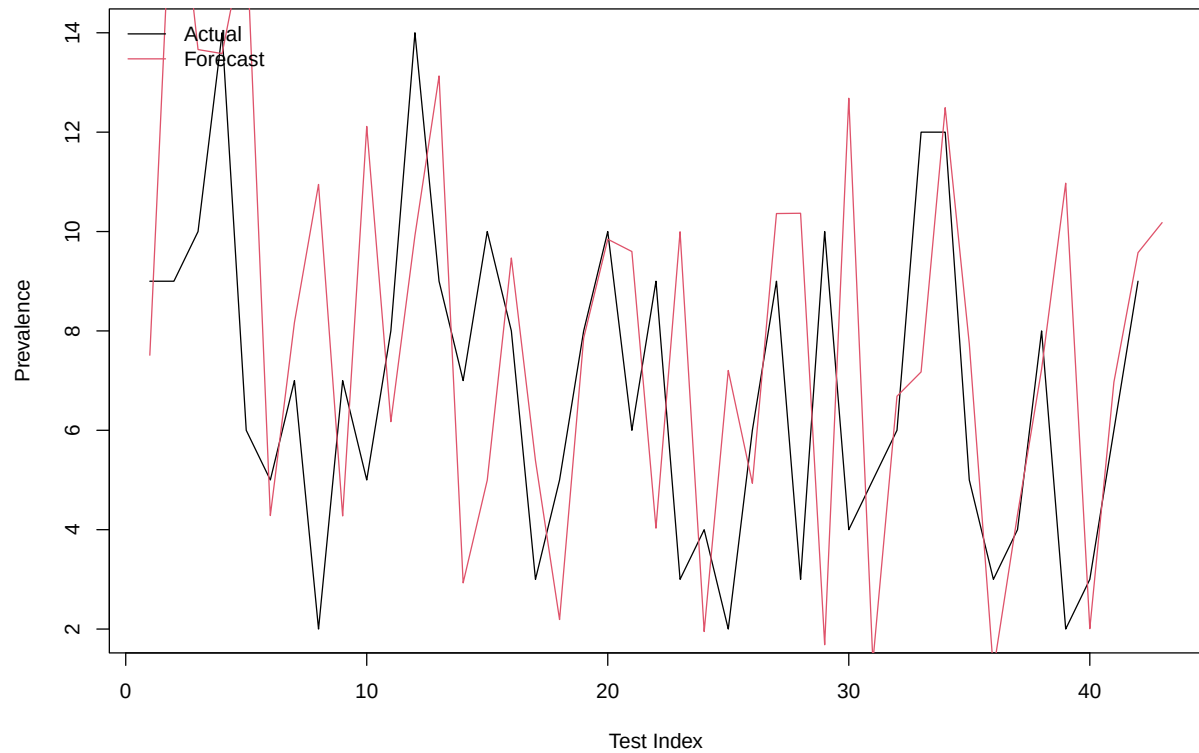
```
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 6ms/epoch - 6ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
```

```

## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 4ms/epoch - 4ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step
## 1/1 - 0s - 5ms/epoch - 5ms/step

```

Test: Actual vs Reconstructed Forecast (Level)



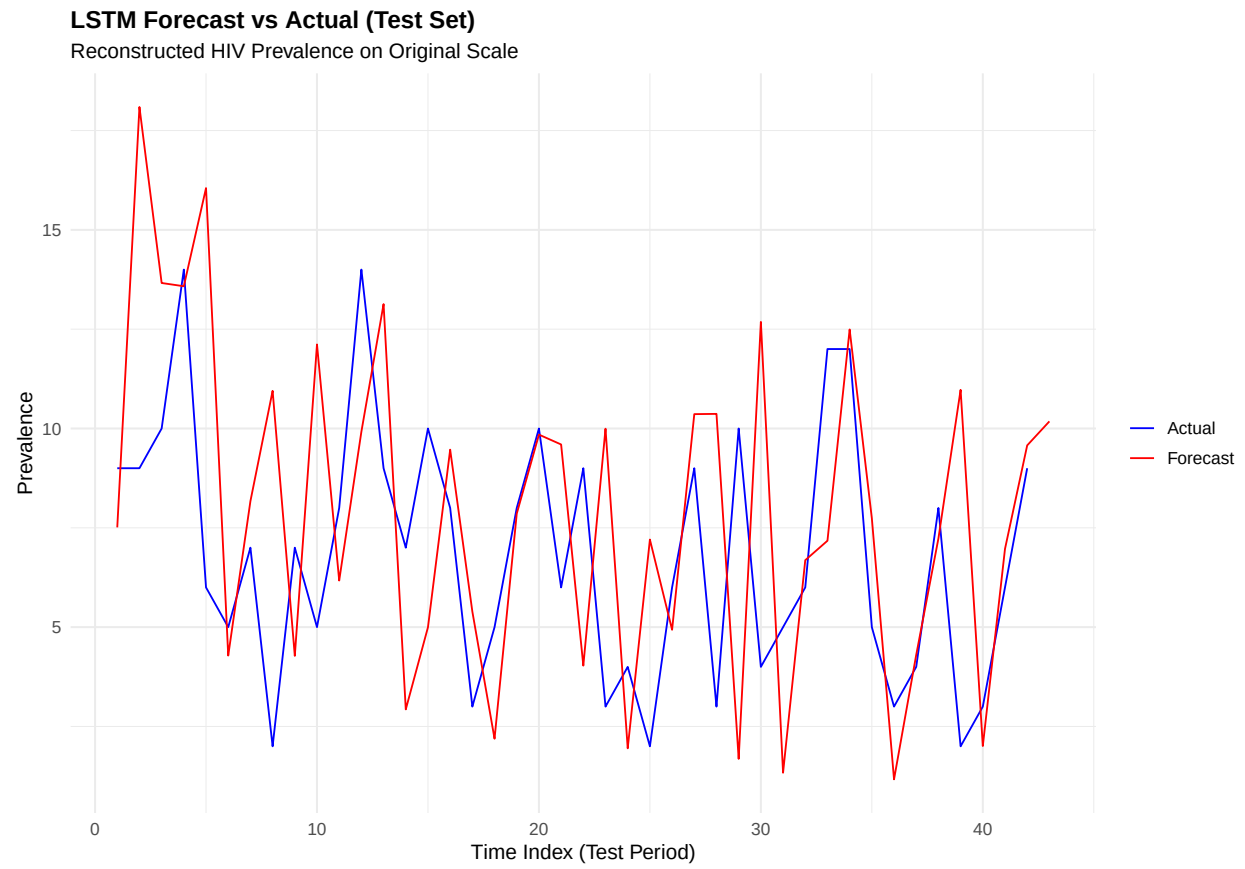


Fig. 1: Data preprocessing pipeline (flowchart)